Computer Vision: **Introduction**

Computer Vision

Building artificial systems that process, perceive, and reason about visual data

Computer Vision is Everywhere

Left to right:
Image by Roger H Goun is licensed under CC BY 2.0
Image is CC0 1.0 public domain
Image is CC0 1.0 public domain
Image is CC0 1.0 public domain

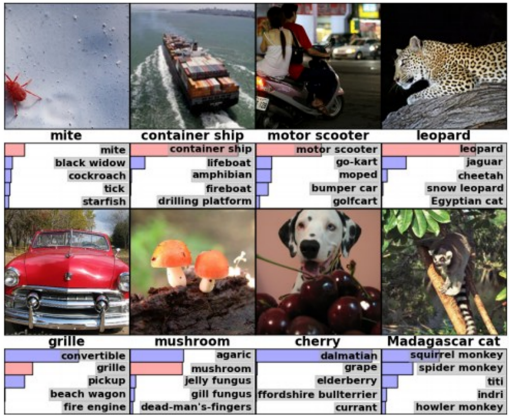
Left to right:
Image is free to use
Image is CC0 1.0 public domain
Image by NASA is licensed under CC BY 2.0
Image is CC0 1.0 public domain

Bottom row, left to right:
Image is CC0 1.0 public domain
Image by Derrek Keats is licensed under CC BY 2.0; changes made
Image is public domain
Image is licensed under CC BY 2.0; changes made

[Source: https://machinelearningmastery.com/stanford-convolutional-neural-networks-for-visual-recognition-course-review/]

Some Applications

Image Classification



Source: Krizhevsky, Sutskever & Hinton, "ImageNet Classification with Deep Convolutional Neural Networks", NIPS 2012, Fig. 4 (left).

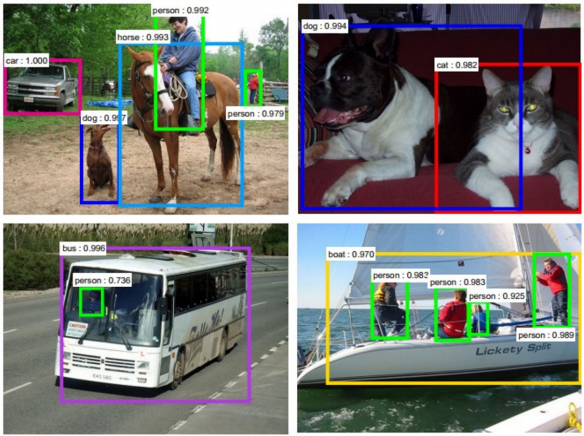
Some Applications (cont.)

Image Retrieval

Source: Krizhevsky, Sutskever & Hinton, "ImageNet Classification with Deep Convolutional Neural Networks", NIPS 2012, Fig. 4 (right).

Some Applications (cont.)

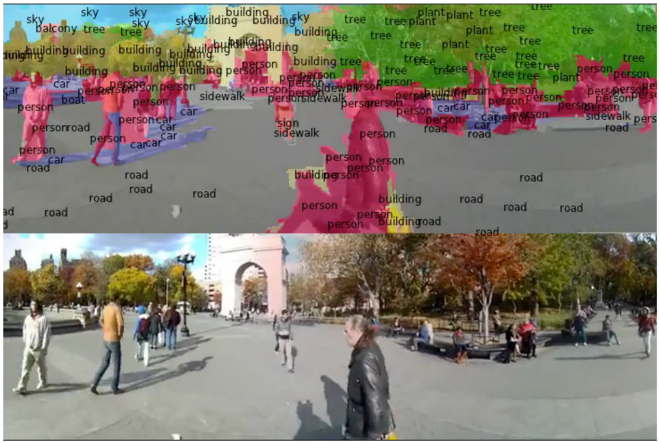
Object Detection



Ren, He, Girshick, and Sun, 2015

Some Applications (cont.)

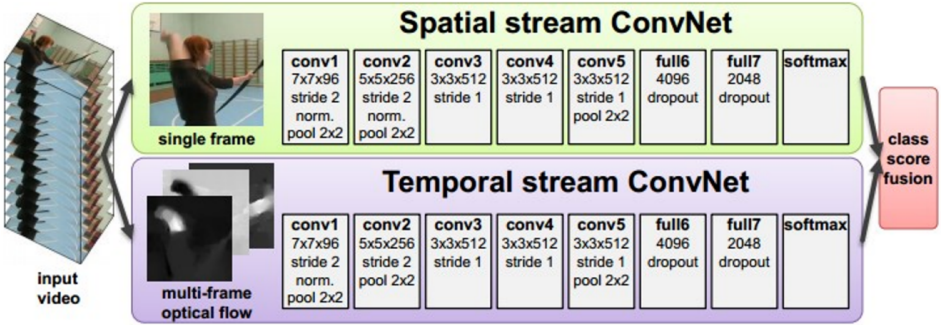
Image Segmentation



Fabaret et al, 2012

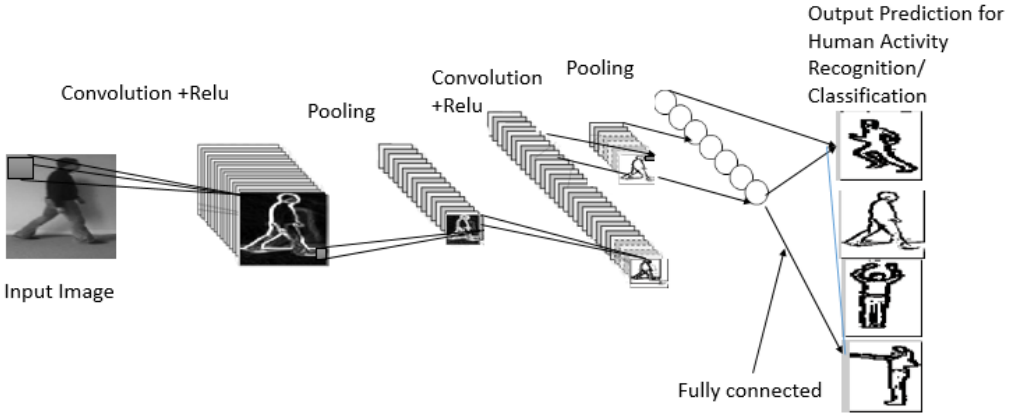
Some Applications (cont.)

Video Classification



Simonyan et al, 2014

Some Applications (cont.)



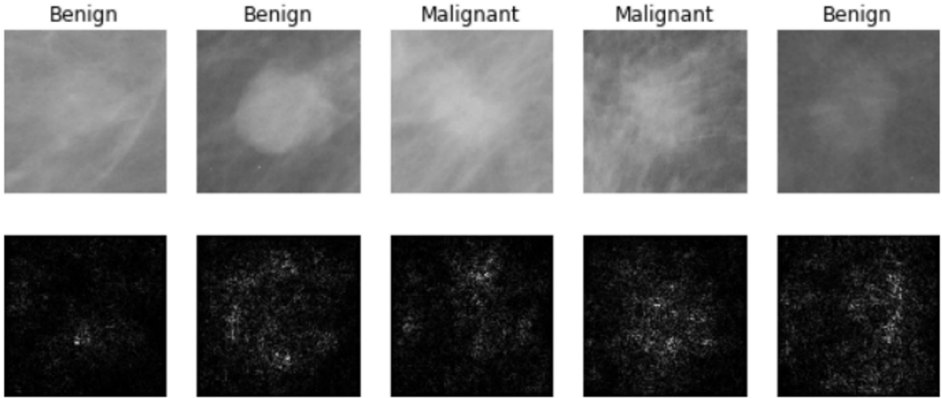
[Source: https://www.researchgate.net/figure/Convolutional-Neural-Network-Architecture-Design-for-Human-Activity-Recognition_fig1_330333654]

Some Applications (cont.)

Pose Recognition (Toshev and Szegedy, 2014)

Some Applications (cont.)

Medical Imaging



[Source: https://ai.stanford.edu/~danilevy/pdf/LevyJain16.pdf]

Some Applications (cont.)

Image Captioning

Vinyals et al, 2015
Karpathy and Fei-Fei, 2015

A white teddy bear sitting in the grass

A man in a baseball uniform throwing a ball

A woman is holding a cat in her hand

A man riding a wave on top of a surfboard

A cat sitting on a suitcase on the floor

A woman standing on a beach holding a surfboard

All images are CC0 Public domain:
https://pixabay.com/en/luggage-antique-cat-1643010/
https://pixabay.com/en/teddy-plush-bears-cute-teddy-bear-1623436/
https://pixabay.com/en/surf-wave-summer-sport-litoral-1668716/
https://pixabay.com/en/woman-female-model-portrait-adult-983967/
https://pixabay.com/en/handstand-lake-meditation-496008/
https://pixabay.com/en/baseball-player-shortstop-infield-1045263/

Some Applications (cont.)

"Teddy bears working on new AI research underwater with 1990s technology"

DALL-E 2

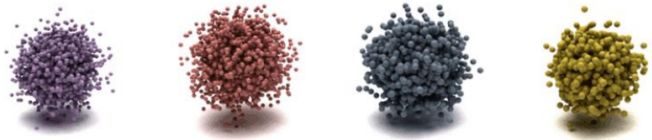
Some Applications (cont.)

Style Transfer

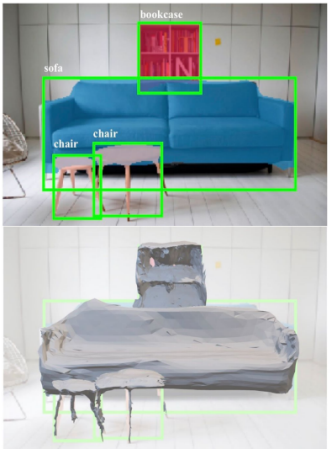
[Source: https://cs231n.stanford.edu/reports/2017/pdfs/401.pdf]

Some Applications (cont.)

Choy et al., 3D-R2N2: Recurrent Reconstruction Neural Network (2016)



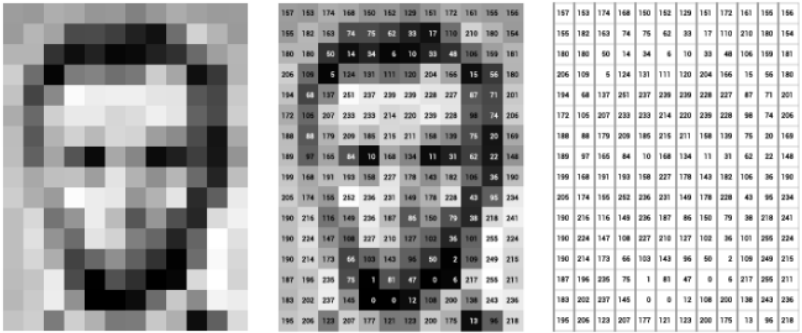
Zhou et al., 3D Shape Generation and Completion through Point-Voxel Diffusion (2021)



Gkioxari et al., "Mesh R-CNN", ICCV 2019

How to represent an image?

- Images are represented as Matrices with elements in $[0, 255]$
- Grayscale images have one channel.



Source: V7 Labs, *Image Recognition: Definition, Algorithms & Uses.*

How to represent an image?

- RGB images have 3 channels.

[Source: https://tech.ardswc.gov.tw/EPaper/Home/EPaper?PaperID=c57d55d5-109a-4319-816b-cc899e65f7a1]

Motivation

- Humans are amazing at understanding images: recognizing faces, reading signs, identifying objects. Computers need to learn patterns in the pixel values instead.
- **Automatic feature learning:** CNNs learn features directly from raw pixels, so we do not have to hand-design edge or texture detectors ourselves.
- **Hierarchy:** early layers pick up edges and colors, later layers combine them into parts and whole objects.
- **Wide applications:** image classification, object detection, facial recognition, medical image analysis, self-driving cars.
- Convolution and CNNs are the backbone of modern computer vision.

Computer Vision: **How Can We Build Vision Models?**

How can we build vision models?

- So far, we've worked with tabular data, where each sample consists of structured features. We processed this data using Fully Connected Neural Networks.

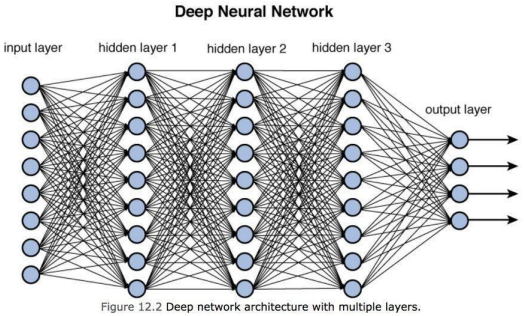


Figure 12.2 Deep network architecture with multiple layers.

[Source: https://www.christophercoverdale.com/blog/neural-networks-101-part-7-data-preprocessing/]

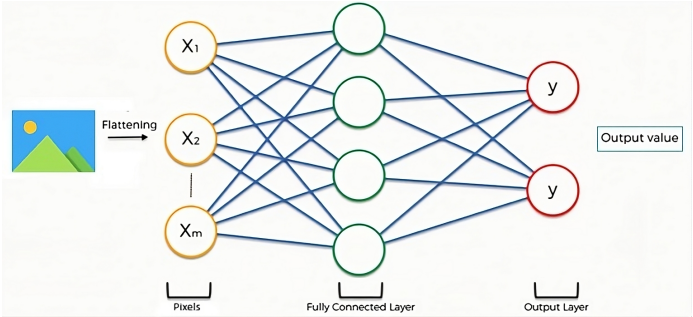
- But can we apply the same approach to images?

How can we build vision models?

- Let's try.

How can we build vision models?

- Let's consider each pixel as a feature.



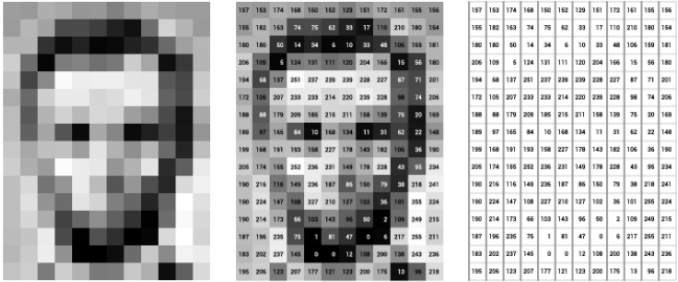
[Source: https://link.springer.com/chapter/10.1007/978-981-97-2977-7_13]

- This should work! But...

CNN: **Introduction to Convolutional Neural Networks**

Why Do We Need Special Models for Images?

Images are grids/matrices of numbers!



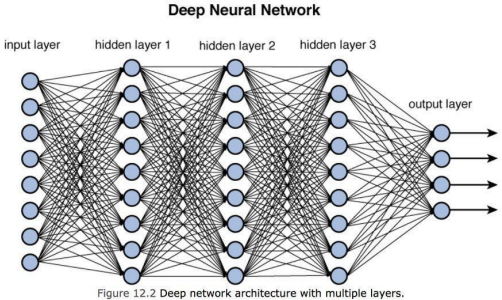
Source: V7 Labs, *Image Recognition: Definition, Algorithms & Uses*.

Images are **BIG** (e.g., $256 \times 256 \times 3 \approx 200{,}000$ numbers)

Why Do We Need Special Models for Images? (cont.)

Fully Connected Neural Networks:

- Too many parameters
- Miss spatial information

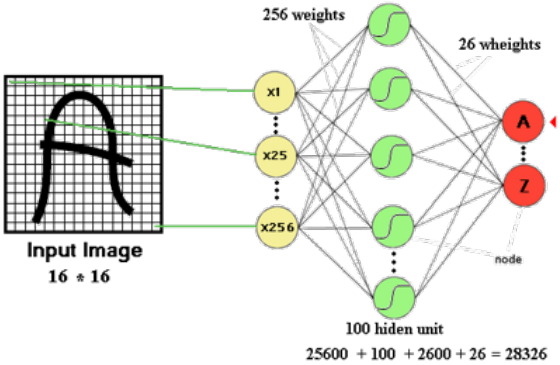


[Source: https://www.christophercoverdale.com/blog/neural-networks-101-part-7-data-preprocessing/]

$$z = W_1 x_1 + W_2 x_2 + \cdots + W_n x_n + b$$

Drawbacks of Fully-Connected Neural Networks

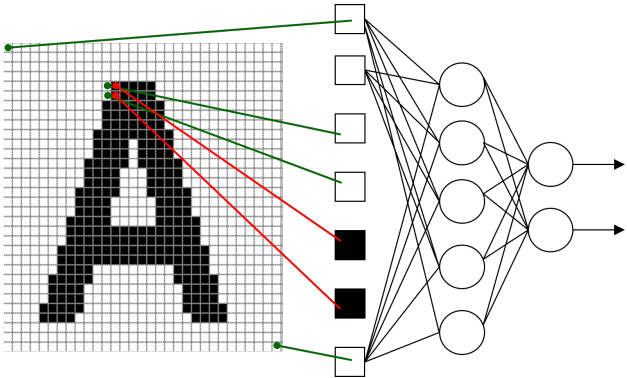
- The number of trainable parameters becomes extremely large



[Source: https://thome.isir.upmc.fr/talks/BSG_THOME_18.pdf]

Drawbacks of Fully-Connected Neural Networks (cont.)

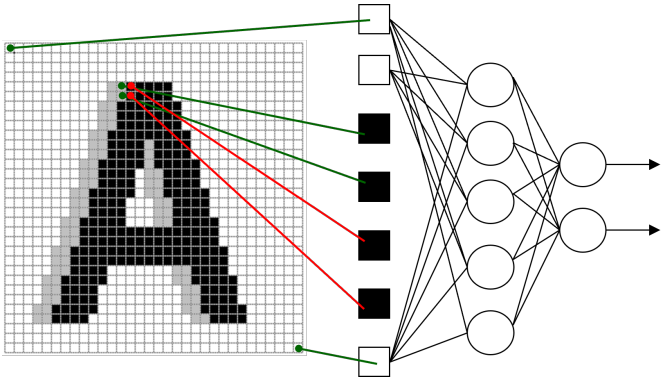
- Little or no invariance to shifting, scaling, and other forms of distortion



[Source: https://medium.com/@Mustafa77/deep-learning-specialization-c4w1-b493f31e5182]

Drawbacks of Fully-Connected Neural Networks (cont.)

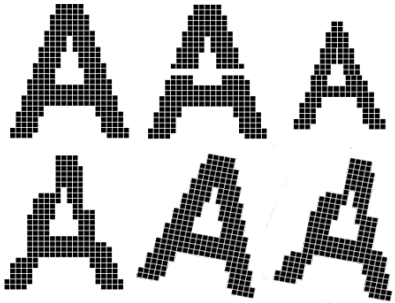
- Little or no invariance to shifting, scaling, and other forms of distortion



[Source: https://medium.com/@Mustafa77/deep-learning-specialization-c4w1-b493f31e5182]

Drawbacks of Fully-Connected Neural Networks (cont.)

- The topology of the input data is completely ignored
- For a 32×32 image, we have
  - Black and white patterns: $2^{32 \times 32} = 2^{1024}$
  - Grayscale patterns: $256^{32 \times 32} = 256^{1024}$



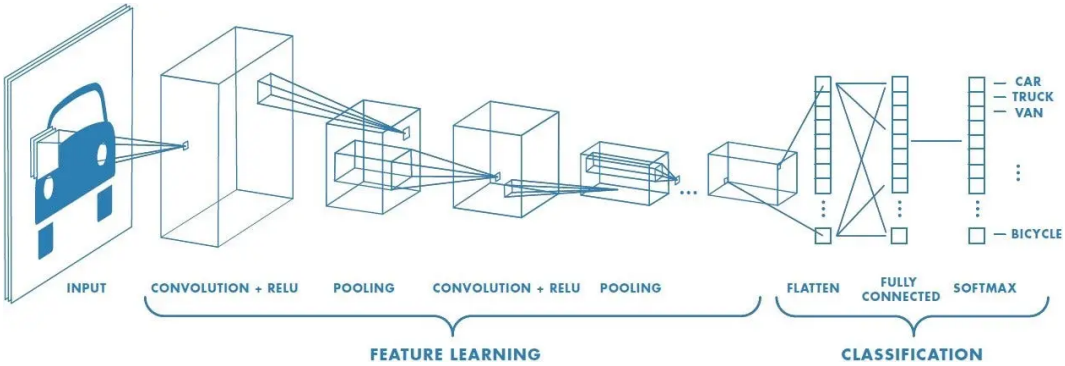
[Source: https://www.hellot.net/news/article_print.html?no=42061]

Drawbacks of Fully-Connected Neural Networks (cont.)

We need something smarter:

Convolution!

Convolutional Neural Networks (CNNs)



Source: MathWorks, *What Is a Convolutional Neural Network?*

$$z_{i,j} = (W * x)_{i,j} = \sum_{a=0}^{m-1} \sum_{b=0}^{n-1} W_{ab} x_{(i+a)(j+b)}$$

What is Convolution?

- A way to process local areas of the image
- Slide a small filter (kernel) over the image
- At each step, do element-wise multiply and sum
- Outputs a feature map (like a filtered image)

What do we need?

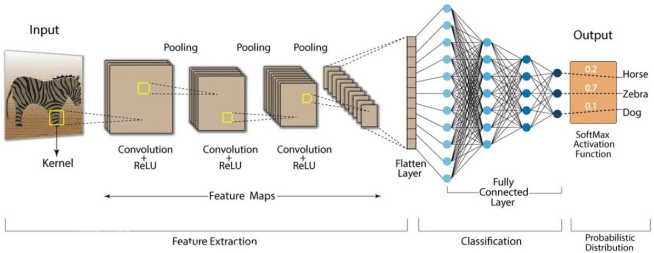
- We need a model that can:
  - **Find Patterns in Images**: Recognize small features like edges and shapes in different parts of the image.
  - **Work Efficiently**: Avoid looking at every pixel separately by focusing on groups of pixels together.
  - **Handle Changes**: Still recognize the same object even if it shifts position in the image (robustness to rotation and scale comes later, from data augmentation).

What do we need?

- We need a model that can:
  - **Find Patterns in Images**: Recognize small features like edges and shapes in different parts of the image.
  - **Work Efficiently**: Avoid looking at every pixel separately by focusing on groups of pixels together.
  - **Handle Changes**: Still recognize the same object even if it shifts position in the image (robustness to rotation and scale comes later, from data augmentation).
- We need **Convolutional Neural Networks (CNNs)!**

Convolutional Neural Networks (CNNs)

- CNNs are neural networks designed for image processing and consist of two main parts:
  - **Feature Extractor**: Automatically learns patterns (features) such as edges, textures, and shapes from the image.
  - **Classifier**: The extracted features are flattened into a vector and passed to a standard neural network (like the ones we used before) for classification.



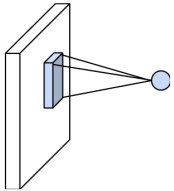
[Source: https://medium.com/@korrabimadjid111/understanding-convolutional-neural-networks-cnns-from-image-to-prediction-79f6c959db4f]

Convolutional Neural Networks (CNNs)

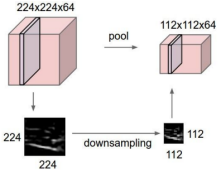
- The feature extractor consists of three essential components:
  - **Convolution Layers**: Detect local patterns such as edges, textures, and shapes by applying filters to the image.
  - **Activation Function (ReLU)**: Adds non-linearity.
  - **Pooling Layers**: Reduce the spatial size of extracted features.
- Let's start with the convolution layer.

Components of a CNN

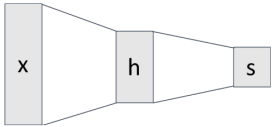
Convolution Layers



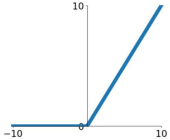
Pooling Layers



Fully-Connected Layers



Activation Function



Normalization

$$\hat{x}_{i,j} = \frac{x_{i,j} - \mu_j}{\sqrt{\sigma_j^2 + \varepsilon}}$$

[Source: https://velog.io/@nameunzz/CS231n-Convolutional-Networks]

CNN: **How Does Convolution Work?**

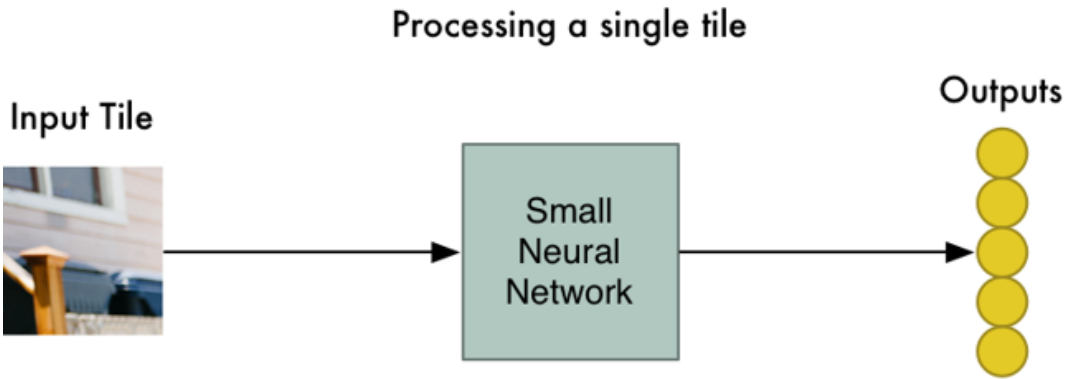
How Does Convolution Work?

[Source: https://www.flickr.com/photos/99146227@N00/]

How Does Convolution Work? (cont.)

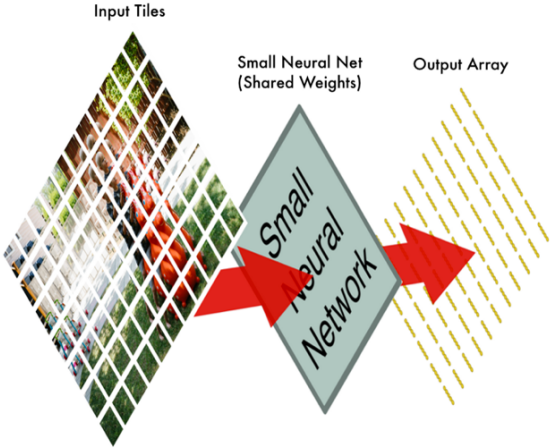
[Source: https://medium.com/the-mission/up-to-speed-on-deep-learning-july-update-part-2-baacc835d8ab]

How Does Convolution Work? (cont.)



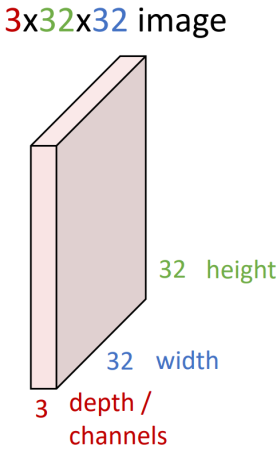
[Source: https://www.researchgate.net/figure/An-input-tile-processed-through-neural-network_fig5_333507222]

How Does Convolution Work? (cont.)



[Source: https://medium.com/@ageitgey/machine-learning-is-fun-part-3-deep-learning-and-convolutional-neural-networks-f40359318721]

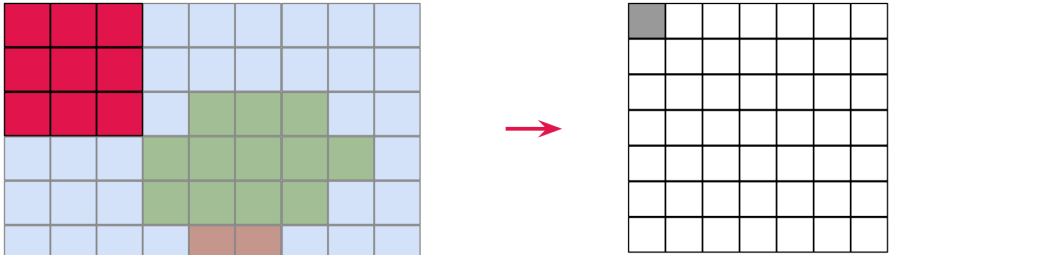
How Does Convolution Work? (cont.)



Convolve the filter with the image i.e. “slide over the image spatially, computing dot products”

[Source: https://cs229.stanford.edu/notes2022fall/cs229-conv_nets_slides.pdf]

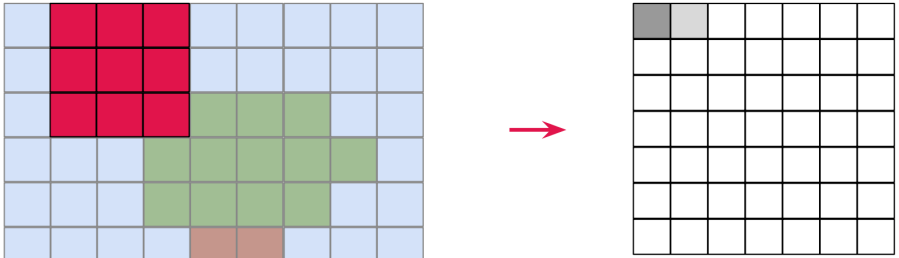
How Does Convolution Work? (cont.)



The **kernel** slides across the image and produces an output value at each position

[Source: https://medium.com/@mohamedhedifkir/convolutional-neural-network-23a6acb08d6a]

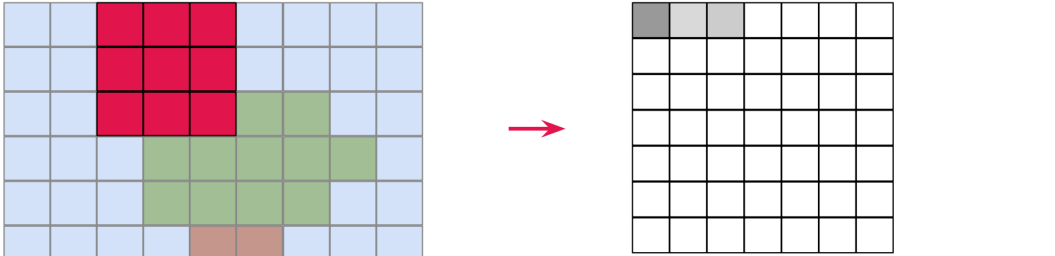
How Does Convolution Work? (cont.)



The **kernel** slides across the image and produces an output value at each position

[Source: https://medium.com/@mohamedhedifkir/convolutional-neural-network-23a6acb08d6a]

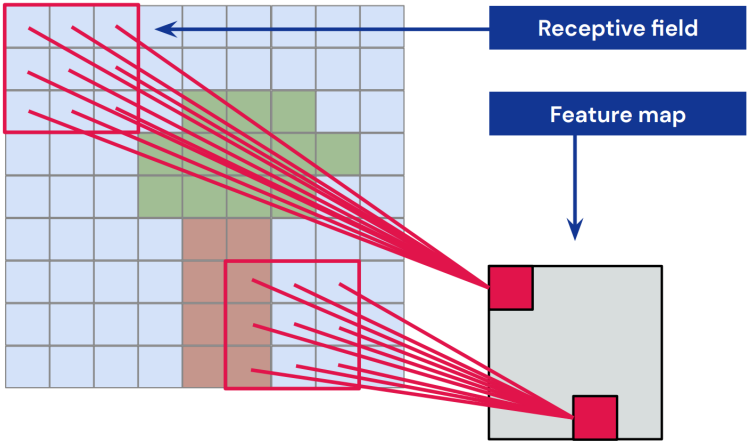
How Does Convolution Work? (cont.)



The **kernel** slides across the image and produces an output value at each position

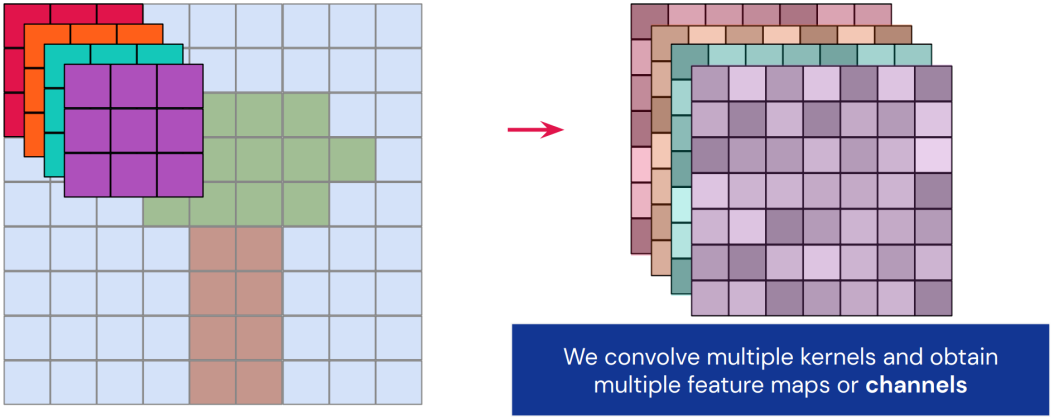
[Source: https://medium.com/@mohamedhedifkir/convolutional-neural-network-23a6acb08d6a]

How Does Convolution Work? (cont.)



[Source:
https://medium.com/@nghihuynh_37300/convolutional-neural-networks-for-image-recognition-7148a19f981f]

How Does Convolution Work? (cont.)

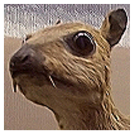


[Source: https://medium.com/@nghihuynh_37300/convolutional-neural-networks-for-image-recognition-7148a19f981f]

How Does Convolution Work? (cont.)

$$\begin{bmatrix} 1 & 0 & -1 \\ 0 & 0 & 0 \\ -1 & 0 & 1 \end{bmatrix} \qquad \begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix}$$

$$\begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix} \qquad \begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix} \qquad \frac{1}{16}\begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$$



[Source: https://medium.com/analytics-vidhya/understanding-convolution-operations-in-cnn-1914045816d4]

CNN Hyperparameter: **Kernels and Filters**

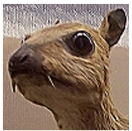
Kernels and Filters

- The terms kernel and filter are often used interchangeably in CNNs; strictly, a filter stacks one 2D kernel per input channel.
- They are small matrices used to extract features from input data.
- The kernel **slides over** the input image, performing **element-wise** multiplication and summation.
- This process is known as **convolution**.

Kernels and Filters (cont.)

$$\begin{bmatrix} 1 & 0 & -1 \\ 0 & 0 & 0 \\ -1 & 0 & 1 \end{bmatrix} \qquad \begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix}$$

$$\begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix} \qquad \begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix} \qquad \frac{1}{16}\begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$$

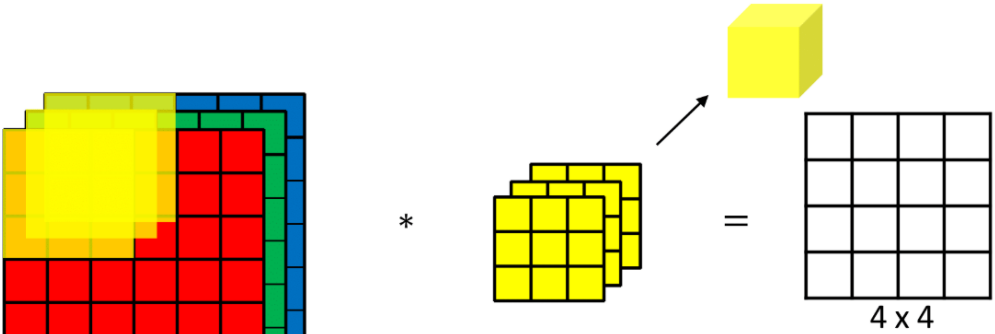


[Source: https://medium.com/analytics-vidhya/understanding-convolution-operations-in-cnn-1914045816d4]

Kernels and Filters (cont.)

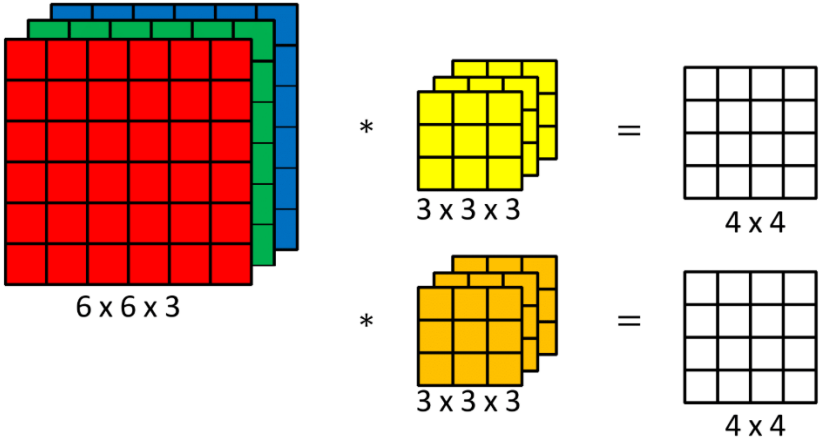
- The kernel size is typically smaller than the input image (e.g., 3x3, 5x5).
- Multiple kernels can be used in a single convolutional layer to extract different features.
- Each kernel produces a feature map, which is a transformed version of the input image highlighting specific features.
- The number of filters (kernels) in a layer determines the depth of the output feature map.

Convolutions on RGB images



[Source: Andrew Ng, *Deep Learning Specialization*, Course 4: Convolutional Neural Networks (Coursera).]

Multiple Channels & Multiple Filters



[Source: Andrew Ng, *Deep Learning Specialization*, Course 4: Convolutional Neural Networks (Coursera).]

Number of Parameters?

If you have 10 filters of size $3 \times 3 \times 3$ in one layer of a neural network, how many parameters does that layer have?

Number of Parameters? (cont.)

Each filter has size $3 \times 3 \times 3 = 27$ parameters (weights).

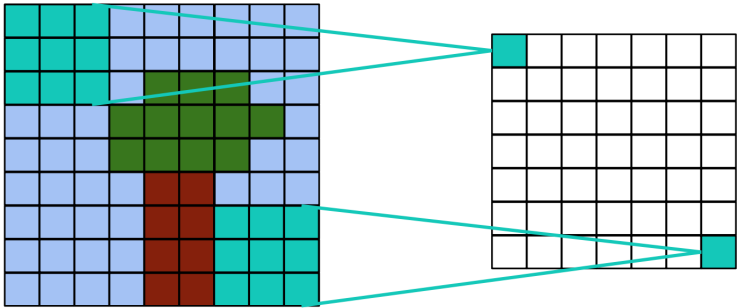
With 10 filters: $10 \times 27 = 270$ weights.

If each filter also has a bias term, add 10 more parameters: $\mathbf{270 + 10 = 280}$ parameters in total.

CNN Hyperparameter: **Padding**

Padding

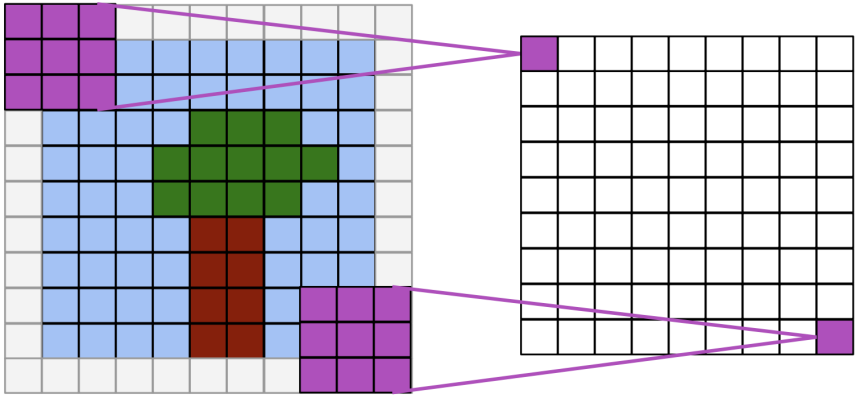
- Applying convolution like this shrinks the output and loses information at the borders.
- Sometimes this is not desirable.
- We can pad the border with zeros.



[Source: https://www.bpesquet.fr/mlhandbook/algorithms/convolutional_neural_networks.html]

Padding (cont.)

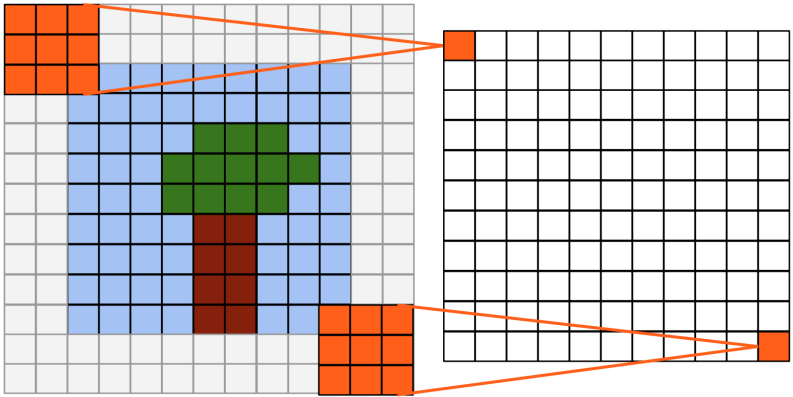
- Same Convolution: Output is the same size as input



[Source: https://www.bpesquet.fr/mlhandbook/algorithms/convolutional_neural_networks.html]

Padding (cont.)

- Full Convolution: output size = input size + kernel size - 1

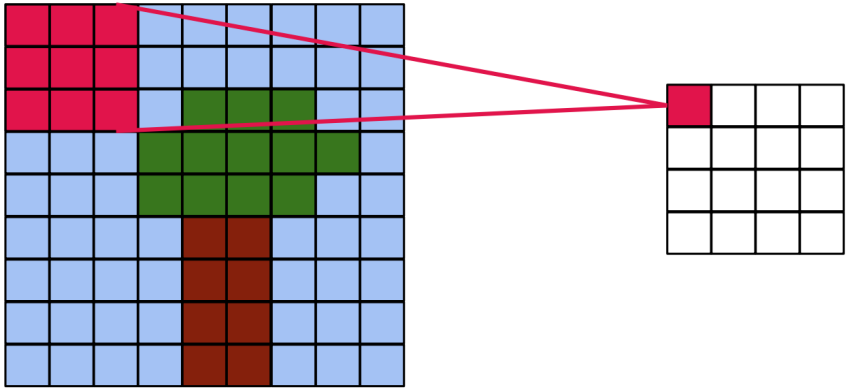


[Source: https://www.bpesquet.fr/mlhandbook/algorithms/convolutional_neural_networks.html]

CNN Hyperparameter: **Strides**

Strided Convolution

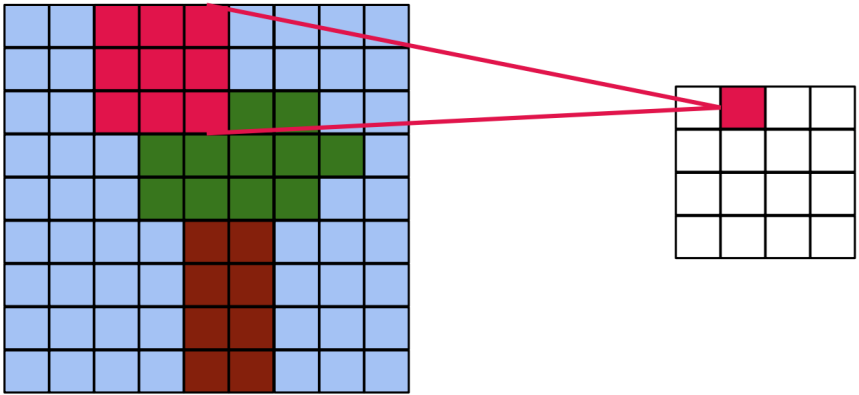
- Kernel slides along the image with a step > 1



[Source: https://medium.com/@mohamedhedifkir/convolutional-neural-network-23a6acb08d6a]

Strided Convolution (cont.)

- Kernel slides along the image with a step > 1



[Source: https://medium.com/@mohamedhedifkir/convolutional-neural-network-23a6acb08d6a]

CNN Hyperparameter: **Padding & Strides Together**

Padding & Strides

Padding:

- "same" preserves spatial size by adding zeros around the input.
- "valid" reduces spatial size by not adding any padding.
- Padding is used to control the spatial size of the output feature map.
- Padding is added to the input image before applying the convolution operation.

Strides:

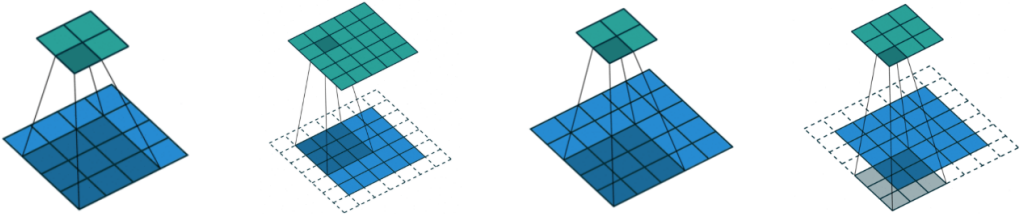
- Strides control how much the filter moves across the input image.
- Strides can be set independently for height and width.
- Strides are used to control the spatial size of the output feature map.
- Strides are set in the convolutional layer.

Listing 1: Code snippet (PyTorch)

```
import torch.nn as nn

conv_valid = nn.Conv2d(3, 16, 3, stride=2, padding=0) # "Valid": no padding (
    padding=0)
conv_same  = nn.Conv2d(3, 16, 3, stride=1, padding=1) # "Same": preserve size
```

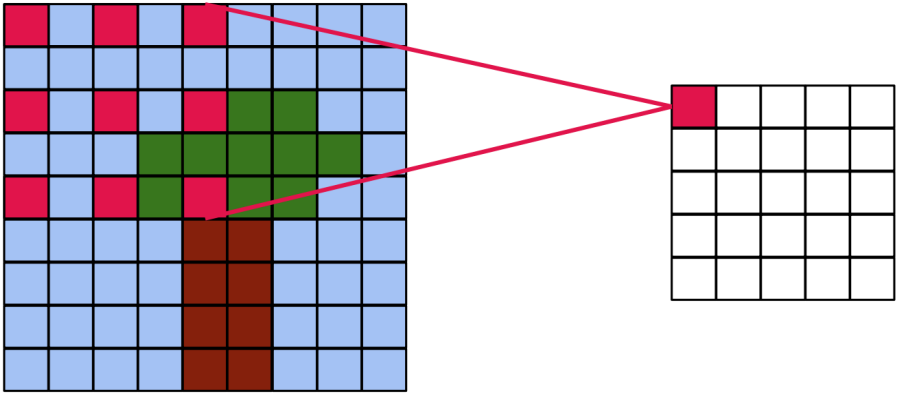
Padding & Strides



Source: Dumoulin & Visin, *A guide to convolution arithmetic for deep learning*, arXiv:1603.07285.

Dilated Convolution

- Kernel is spread out, step > 1 between kernel elements



[Source: https://medium.com/@mohamedhedifkir/convolutional-neural-network-23a6acb08d6a]

Output Shape

- Output shape can be calculated as:

$$O = \left\lfloor \frac{I + 2P - K}{S} + 1 \right\rfloor$$

where:

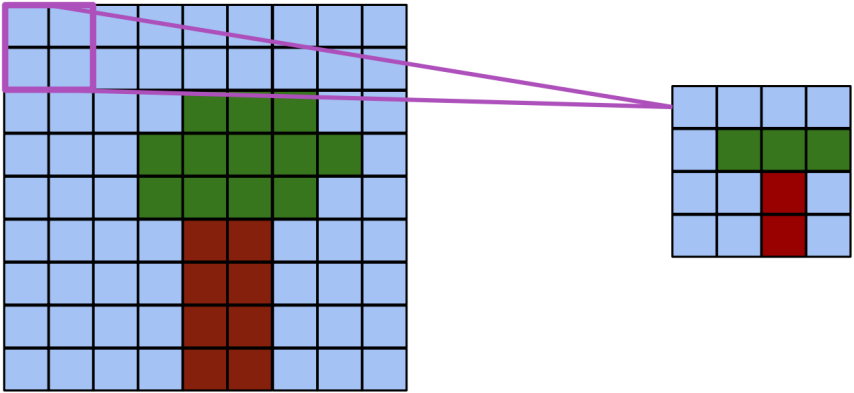
- O = output size
- I = input size
- P = padding
- K = kernel size
- S = stride

- This assumes a dilation of 1. With dilation D, replace K by the effective kernel size $K' = D(K-1)+1$, so a dilated 3×3 kernel with $D = 2$ spans a 5×5 area.

CNN Layer: **Pooling Layers**

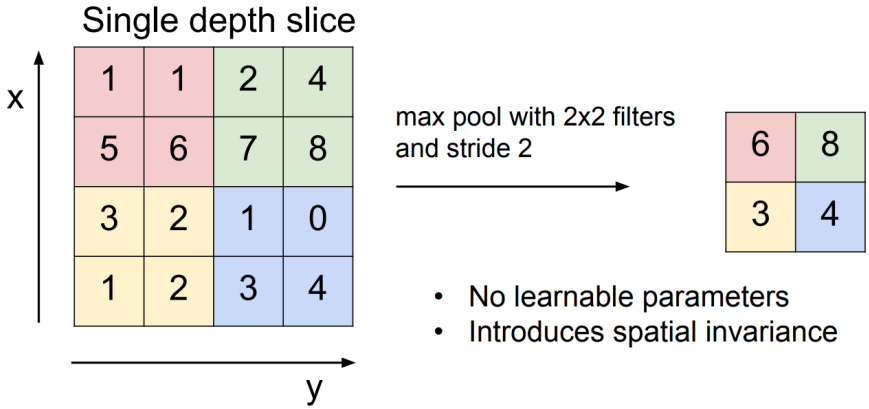
Pooling

- Compute mean or max over small windows to reduce resolution



[Source: https://medium.com/@mohamedhedifkir/convolutional-neural-network-23a6acb08d6a]

Pooling (cont.)



- No learnable parameters
- Introduces spatial invariance

[Source: https://medium.com/@mohamedhedifkir/convolutional-neural-network-23a6acb08d6a]

Pooling Layers

Purpose:

- Downsample feature maps, reduce spatial dims and parameters, add tolerance to small shifts.

Types:

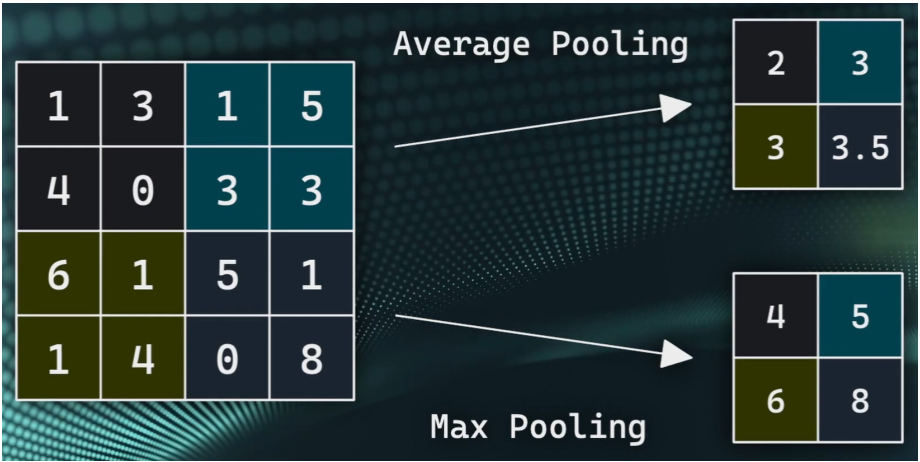
- Max Pooling
- Average Pooling
- Global Average Pooling
- Global Max Pooling

Listing 2: Code snippet (PyTorch)

```
import torch.nn as nn

pool = nn.MaxPool2d(kernel_size=2, stride=2)
pooled = pool(x)  # halves H and W
```

Pooling Layers

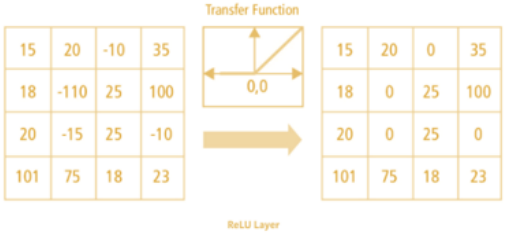


[Source: https://www.picsellia.com/post/image-embeddings-explained]

CNN Layer: **Activation Functions**

Activation

- Just like Fully-Connected Neural Networks, we can apply an activation over convolutional layer outputs
- It helps break linearity
- For example, Rectified Linear Unit (ReLU): $f(x) = \max(0, x)$



[Source: https://medium.com/@nathanbendich/introduction-to-convolutional-neural-networks-deep-learning-12febe285b12]

Activation Functions

Role:

- Introduce non-linearity so multiple conv layers can learn complex mappings.

Common:

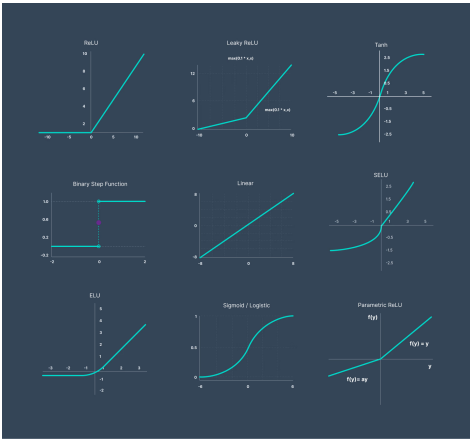
- ReLU (Rectified Linear Unit)
- Leaky ReLU
- Sigmoid
- Tanh

Listing 3: Code snippet (PyTorch)

```
import torch.nn.functional as F

x = conv(input_tensor)
x = F.relu(x)                # ReLU
x = F.leaky_relu(x, 0.1)     # Leaky ReLU
```

Activation Functions



[Source: https://medium.com/@vidixha/from-neurons-to-networks-ep-2-part-get-set-activate-activation-functions-1627c46984d4]

CNN: **Flattening & Fully Connected Layers**

Flattening & Fully Connected Layers

Flatten:

- Flattening is the process of converting a multi-dimensional tensor into a one-dimensional tensor.
- Flattening is typically done before passing the data to fully connected layers.

FC Layer:

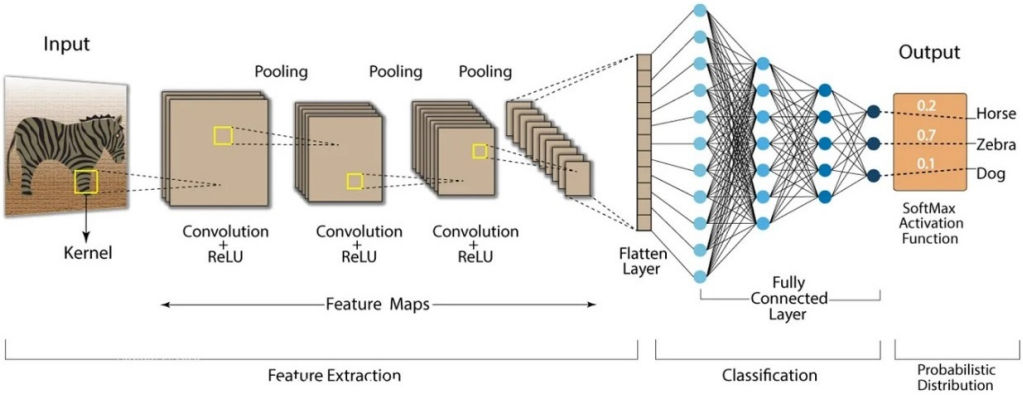
- Used to learn complex relationships between features.
- Typically used at the end of the network to make predictions.
- Takes the flattened output from the previous layer and produces a vector of class scores.
- Is often followed by activation functions like ReLU or softmax.

Listing 4: Code snippet (PyTorch)

```
import torch.nn as nn
import torch

x = torch.flatten(x, 1)  # preserve batch dim
fc = nn.Linear(in_features=16*8*8, out_features=10)
out = fc(x)
```

Flattening & Fully Connected Layers



[Source: https://medium.com/@korrabimadjid111/understanding-convolutional-neural-networks-cnns-from-image-to-prediction-79f6c959db4f]

CNN: **Putting It Together**

Tracing Shapes Through a Small CNN

Take a $32 \times 32 \times 3$ input and apply $O = \left\lfloor \frac{I+2P-K}{S} + 1 \right\rfloor$ at every step.

Layer	Setting	Output shape
Input		$32 \times 32 \times 3$
Convolution + ReLU	32 filters, $K = 3$, $P = 1$, $S = 1$	$32 \times 32 \times 32$
Max pooling	$K = 2$, $S = 2$	$16 \times 16 \times 32$
Convolution + ReLU	64 filters, $K = 3$, $P = 1$, $S = 1$	$16 \times 16 \times 64$
Max pooling	$K = 2$, $S = 2$	$8 \times 8 \times 64$
Flatten		4096
Fully connected + ReLU	128 units	128
Fully connected	10 classes	10

- Padding of 1 with a 3×3 kernel keeps the resolution: $\lfloor (32 + 2 - 3)/1 \rfloor + 1 = 32$.
- Each pooling layer halves height and width: $\lfloor (32 - 2)/2 \rfloor + 1 = 16$.
- Depth is set by the number of filters, resolution by padding and stride.

Tracing Shapes Through a Small CNN (cont.)

Where do the parameters live?

- Convolution 1: $3 \times 3 \times 3 \times 32 + 32 = 896$
- Convolution 2: $3 \times 3 \times 32 \times 64 + 64 = 18{,}496$
- Fully connected 1: $4096 \times 128 + 128 = 524{,}416$
- Fully connected 2: $128 \times 10 + 10 = 1{,}290$
- Total: 545,098

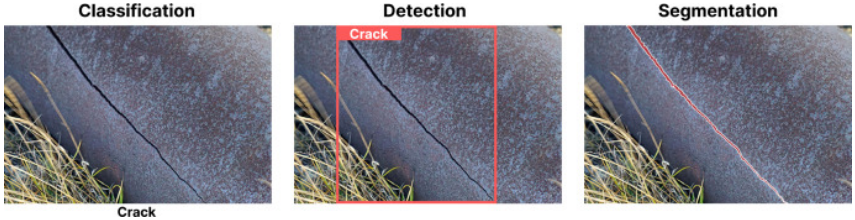
- The two convolution layers hold 19,392 weights, about 3.6% of the network. The first fully connected layer alone holds 96%.
- Convolution is cheap because the same kernel is reused at every position. A fully connected layer needs its own weight for every input.
- This is why architectures covered later shrink the feature map further before flattening, or drop the flatten step and use global average pooling instead.

Computer Vision: **Industry Use Cases**

Pipeline Corrosion & Defect Analysis

- **CNN Architectures Used:**
  - ResNet
  - VGG
  - InceptionNet
- **Use Case:**
  - Automated analysis of pipeline conditions using camera feeds.
  - Classification of corrosion severity levels.
  - Predictive maintenance through visual pattern recognition.

Pipeline Corrosion & Defect Analysis (cont.)



[Source: https://www.sciencedirect.com/science/article/pii/S2667143324000738]

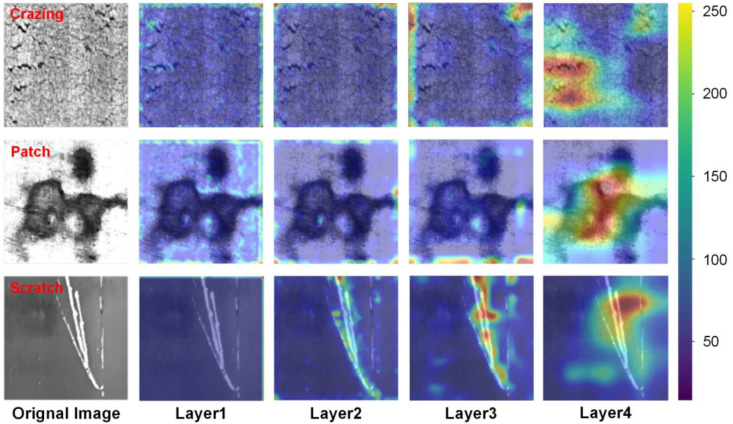
Pipeline Corrosion & Defect Analysis (cont.)

- **Benefits:**
  - Reduces manual inspection costs and time.
  - Early detection prevents catastrophic failures.
  - Enables data-driven maintenance scheduling.
- **Links & Resources:**
  - viso.ai
  - ScienceDirect
  - NextBrain

Equipment Defect Classification

- **CNN Architectures Used:**
  - ResNet50 with Transfer Learning
  - VGG19
  - AlexNet
- **Use Case:**
  - Classify types of defects in steel surfaces and equipment.
  - Automated quality control for manufactured components.
  - Multi-class defect recognition (rust, cracks, deformation, etc.).

Equipment Defect Classification (cont.)



[Source: https://www.mdpi.com/2076-3417/13/9/5260]

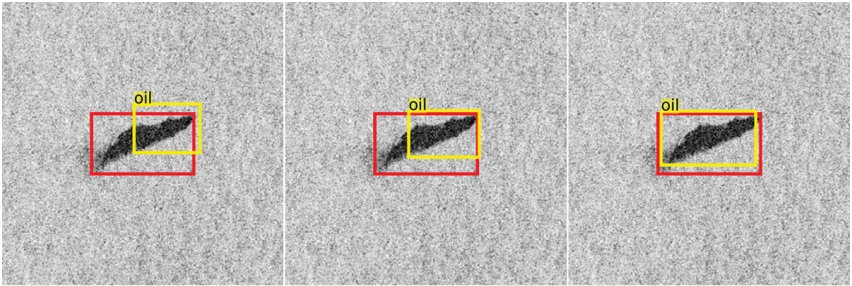
Equipment Defect Classification (cont.)

- **Benefits:**
  - 99.4% accuracy with transfer learning approaches.
  - Cost savings through early detection.
  - Replaces subjective manual inspections.
- **Links & Resources:**
  - MDPI: Steel defect detection (ResNet, 99.4% accuracy)

Oil Spill Detection & Classification

- **CNN Architectures Used:**
  - YOLOv4
  - Mask R-CNN
  - CNNs
- **Use Case:**
  - Classify oil spill severity and type from SAR/optical imagery.
  - Automated environmental monitoring systems.
  - Multi-class classification for response prioritization.

Oil Spill Detection & Classification (cont.)



(a) IoU = 30% (b) IoU = 45% (c) IoU = 75%

[Source: https://www.tandfonline.com/doi/full/10.1080/01431161.2022.2109445]

Oil Spill Detection & Classification (cont.)

- **Benefits:**
  - Faster emergency response decisions.
  - Accurate spill characterization for cleanup planning.
  - Compliance with environmental regulations.
- **Research Papers & Resources:**
  - Deep learning oil spill detector (Sentinel-1 SAR, 2022)
  - Review of computer vision for asset inspection in oil & gas (2024)